



CSF 302

Software Engineering Department

Lab Report 2

<Submitted by>

Student No:02240337

**Royal University of Bhutan, College of Science and
Technology**

Q1: Prime Number Testing

Objective:

Test whether a number n is prime using two algorithms and compare their performance.

Algorithms:

1. Naive Method (check divisibility from 2 to $n-1$)
2. Optimized Method (check divisibility up to $\sqrt{n}$)
3. OPTIONAL : Find the list of primes using the sieve of Eratosthenes algorithm.

Instructions:

- Input at least 10 numbers from the user.
- For each number, determine if it is prime using both algorithms.
- Record step/frequency counts for each algorithm.
- Compare time complexities and plot a graph to identify the faster algorithm.

Output:

```
> PS D:\3rd Year 5th Sem\CSF302_AS2026_02248337\Lab Practicals\Lab2_AS2026> C:\Users\User\AppData\Local\Programs\Python\Python310\python.exe "d:/3rd Year 5th Sem/CSF302_AS2026_02248337/Lab Practicals/La
b2_AS2026/prime_test.py"
Enter at least 10 numbers:
Number 1: 22
Number 2: 76
Number 3: 98
Number 4: 34
Number 5: 777
Number 6: 991
Number 7: 568
Number 8: 2
Number 9: 1
Number 10: 1000
22: Naive=False(1 steps), Optimized=False(1 steps)
76: Naive=False(1 steps), Optimized=False(1 steps)
98: Naive=False(1 steps), Optimized=False(1 steps)
34: Naive=False(1 steps), Optimized=False(1 steps)
777: Naive=False(2 steps), Optimized=False(2 steps)
991: Naive=True(989 steps), Optimized=True(30 steps)
568: Naive=False(1 steps), Optimized=False(1 steps)
2: Naive=True(0 steps), Optimized=True(0 steps)
1: Naive=False(0 steps), Optimized=False(0 steps)
1000: Naive=False(1 steps), Optimized=False(1 steps)

Avg Time: Naive=0.010ms, Opt=0.003ms
Speedup: 3.35x faster
```

Code:

```
import math, time, matplotlib.pyplot as plt

def naive_prime_337(n):
    if n < 2: return False, 0
    steps = 0
    for i in range(2, n):
        steps += 1
        if n % i == 0: return False, steps
    return True, steps

def optimized_prime_337(n):
    if n < 2: return False, 0
    steps = 0
    for i in range(2, int(math.sqrt(n)) + 1):
        steps += 1
        if n % i == 0: return False, steps
    return True, steps

numbers_337 = []
print("Enter at least 10 numbers:")
while len(numbers_337) < 10:
    try:
        numbers_337.append(int(input(f"Number {len(numbers_337)+1}: ")))
    except: print("Invalid input")

naive_times_337, opt_times_337 = [], []
naive_steps_337, opt_steps_337 = [], []

for n in numbers_337:
    start = time.perf_counter()
    is_naive, s1 = naive_prime_337(n)
    naive_times_337.append((time.perf_counter() - start) * 1000)
    naive_steps_337.append(s1)

    start = time.perf_counter()
    is_opt, s2 = optimized_prime_337(n)
    opt_times_337.append((time.perf_counter() - start) * 1000)
    opt_steps_337.append(s2)

    print(f"{n}: Naive={is_naive}({s1} steps), Optimized={is_opt}({s2} steps)")

print(f"\nAvg Time: Naive=(sum(naive_times_337)/len(naive_times_337):.3f)ms, Opt=(sum(opt_times_337)/len(opt_times_337):.3f)ms")
print(f"Speedup: (sum(naive_times_337)/sum(opt_times_337):.2f)x faster")
```

```
# Q1: Prime Number Testing - Analysis & Conclusion
# The optimized method (checking up to  $\sqrt{n}$ ) is significantly faster than the naive method (checking up to  $n-1$ ).
# Time complexity: Naive =  $O(n)$ , Optimized =  $O(\sqrt{n})$ 
# The step count reduction is approximately  $\sqrt{n}$  times, making it much more efficient for larger numbers.
# For prime numbers, the difference is most noticeable as the naive method checks all numbers up to  $n-1$ .
```

Q2: Merge Sort Analysis (Menu-Driven Program)

Objective:

Write a menu driven program to Sort an array of n integers in ascending order using Merge Sort and analyze step/frequency counts for different input types.

Menu:

1. Generate n random numbers $\rightarrow$ Array
2. Display Array
3. Sort in Ascending Order using Merge Sort
4. Sort in Descending Order using any sorting algorithm
5. Time Complexity for ascending random data
6. Time Complexity for ascending already sorted data
7. Time Complexity for ascending descending-sorted data

Instructions:

- Record step/frequency counts.

Output:

```

PS D:\3rd Year 5th Sem\CSF302_AS2026_02240337\Lab Practicals\Lab2_AS2026> & C:\Users\User\AppData\Local\Programs\Python\Python310\python.exe "d:/3rd Year 5th Sem/CSF302_AS2026_02240337/Lab Practicals/Lab2_AS2026/merge_sort.py"

1.Generate 2.Display 3.Ascend(Merge) 4.Descend 5.Random Time 6.Sorted Time 7.Descending Time 0.Exit
Choice: 1
n: 7
Generated 7 numbers

1.Generate 2.Display 3.Ascend(Merge) 4.Descend 5.Random Time 6.Sorted Time 7.Descending Time 0.Exit
Choice: 2
Array: [618, 646, 580, 611, 62, 266, 918]

1.Generate 2.Display 3.Ascend(Merge) 4.Descend 5.Random Time 6.Sorted Time 7.Descending Time 0.Exit
Choice: 3
Sorted: [62, 266, 580, 611, 618, 646, 918], Steps: 14

1.Generate 2.Display 3.Ascend(Merge) 4.Descend 5.Random Time 6.Sorted Time 7.Descending Time 0.Exit
Choice: 4
Descending: [918, 646, 618, 611, 580, 266, 62]

1.Generate 2.Display 3.Ascend(Merge) 4.Descend 5.Random Time 6.Sorted Time 7.Descending Time 0.Exit
Choice: 5

Random Data Analysis_337
Size      Steps      Time(ms)
100       544         0.174
500       3856        1.189
1000      8728        1.849
2000     19464        4.032
3000     30966        6.684

1.Generate 2.Display 3.Ascend(Merge) 4.Descend 5.Random Time 6.Sorted Time 7.Descending Time 0.Exit
Choice: 6

Sorted Data Analysis_337
Size      Steps      Time(ms)
100       316         0.137
500       2216        0.679
1000      4932        1.173
2000     10864        2.850
3000     16828        3.984

1.Generate 2.Display 3.Ascend(Merge) 4.Descend 5.Random Time 6.Sorted Time 7.Descending Time 0.Exit
Choice: 7

Descending Data Analysis_337
Size      Steps      Time(ms)
100       356         0.162
500       2272        0.675
1000      5804        1.674
2000     11888        3.176
3000     18078        4.554

1.Generate 2.Display 3.Ascend(Merge) 4.Descend 5.Random Time 6.Sorted Time 7.Descending Time 0.Exit
Choice: 0
PS D:\3rd Year 5th Sem\CSF302_AS2026_02240337\Lab Practicals\Lab2_AS2026>

```

Code:

```

import random, time

def merge_sort_337(arr, steps):
    if len(arr) <= 1: return arr, steps
    mid = len(arr)//2
    left, steps = merge_sort_337(arr[:mid], steps)
    right, steps = merge_sort_337(arr[mid:], steps)
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        steps += 1
        if left[i] <= right[j]: result.append(left[i]); i += 1
        else: result.append(right[j]); j += 1
    return result + left[i:] + right[j:], steps

array_337, n_337, steps_337 = [], 0, 0

while True:
    print("\n1.Generate 2.Display 3.Ascend(Merge) 4.Descend 5.Random Time 6.Sorted Time 7.Descending Time 0.Exit")
    choice = input("Choice: ")

    if choice == '1':
        n_337 = int(input("n: "))
        array_337 = [random.randint(1,1000) for _ in range(n_337)]
        print(f"Generated {n_337} numbers")

    elif choice == '2': print(f"Array: {array_337}")

    elif choice == '3':
        if array_337:
            array_337, steps_337 = merge_sort_337(array_337, 0)
            print(f"Sorted: {array_337}, Steps: {steps_337}")
        else: print("Empty!")

    elif choice == '4':
        if array_337: array_337.sort(reverse=True); print(f"Descending: {array_337}")
        else: print("Empty!")

```

```

    elif choice == '4':
        if array_337: array_337.sort(reverse=True); print(f"Descending: {array_337}")
        else: print("Empty!")

    elif choice in ['5','6','7']:
        labels = {'5':'Random','6':'Sorted','7':'Descending'}
        print(f"\n{n(labels[choice])} Data Analysis_337")
        print(f"{'Size':<10}{Steps':<15}{Time(ms)':<15}")
        for size in [100,500,1000,2000,3000]:
            if choice == '5': arr = [random.randint(1,1000) for _ in range(size)]
            elif choice == '6': arr = list(range(size))
            else: arr = list(range(size,0,-1))
            start = time.perf_counter()
            _, steps = merge_sort_337(arr, 0)
            elapsed = (time.perf_counter() - start) * 1000
            print(f"{'size':<10}{steps:<15}{elapsed:<15.3f}")

    elif choice == '0': break
    else: print("Invalid choice!")

```

Q2: Merge Sort Analysis - Analysis & Conclusion

Merge Sort consistently shows $O(n \log n)$ time complexity regardless of input order. Step counts remain relatively stable for random, sorted, and reverse-sorted data. This demonstrates Merge Sort's stability and predictable performance. The algorithm efficiently handles all input types with the same time complexity. Best/Average/Worst case: $O(n \log n)$ - making it suitable for large datasets.

Q3: Square Matrix Multiplication

Objective:

Multiply two square matrices ($n \times n$), Note: make the matrix as a power of 2, eg, 2×2 , 4×4 , 8×8 ...etc.

Instructions:

- Input matrices manually or generate randomly.
- Display the resultant matrix.
- Count the number of basic operations (step/frequency count).

Output:

```
C:\Users\User\AppData\Local\Programs\Python\Python39\python.exe -u -x --year 5th Sem/CSE2026-02240337/Lab Practicals/Lab_2026-02240337/Matrices/Matrix_Mult.py
Power of 2 (e.g., 2 for 4x4): 2
Generated 4x4 matrices

1.Generate 2.Enter 3.Display 4.Multiply 5.Analysis 0.Exit
Choice: 2
Size (power of 2): 2
Matrix A (space separated rows):
1 2
3 4
Matrix B:
5 6
7 8

1.Generate 2.Enter 3.Display 4.Multiply 5.Analysis 0.Exit
Choice: 4

Result:
[19, 22]
[43, 50]
Operations: 8

1.Generate 2.Enter 3.Display 4.Multiply 5.Analysis 0.Exit
Choice: 5

Size      Operations
2x2       8
4x4       64
8x8      512
16x16    4096
32x32   32768
Theoretical: O(n^3)
```

Code:

```

import random

def multiply_337(A, B, n):
    C = [[0]*n for _ in range(n)]
    steps = 0
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
                steps += 1
    return C, steps

def print_mat_337(M, name):
    print(f"\n{name}:")
    for row in M: print(row)

A, B, n = [], [], 0

while True:
    print("\n1.Generate 2.Enter 3.Display 4.Multiply 5.Analysis 0.Exit")
    choice = input("Choice: ")

    if choice == '1':
        p = int(input("Power of 2 (e.g., 2 for 4x4): "))
        n = 2 ** p
        A = [[random.randint(1,10) for _ in range(n)] for _ in range(n)]
        B = [[random.randint(1,10) for _ in range(n)] for _ in range(n)]
        print(f"Generated {n}x{n} matrices")

    elif choice == '2':
        n = int(input("Size (power of 2): "))
        print("Matrix A (space separated rows):")
        A = [list(map(int, input().split())) for _ in range(n)]
        print("Matrix B:")
        B = [list(map(int, input().split())) for _ in range(n)]

    elif choice == '3':
        if A and B: print_mat_337(A, "A"); print_mat_337(B, "B")
        else: print("Empty!")

    elif choice == '4':
        if A and B:
            C, steps = multiply_337(A, B, n)
            print_mat_337(C, "Result")
            print(f"Operations: {steps}")
        else: print("Empty!")

    elif choice == '5':
        print(f"\n{'Size':<10}{ 'Operations':<20}")
        for p in range(1,6):
            size = 2 ** p
            At = [[random.randint(1,10) for _ in range(size)] for _ in range(size)]
            Bt = [[random.randint(1,10) for _ in range(size)] for _ in range(size)]
            _, ops = multiply_337(At, Bt, size)
            print(f"{size}x{size:<7}{ops:<20}")
        print("Theoretical: O(n³)")

    elif choice == '0': break
    else: print("Invalid!")

# Q3: Square Matrix Multiplication - Analysis & Conclusion
The standard matrix multiplication algorithm has  $O(n^3)$  time complexity.
Step count grows cubically with matrix size ( $n^3$  operations).
For power of 2 matrices (2x2, 4x4, 8x8, 16x16, 32x32), operations =  $n^3$  exactly.
Performance degrades quickly for larger matrices due to cubic complexity.
Optimization techniques like Strassen's algorithm ( $O(n^{2.81})$ ) could improve performance.

```